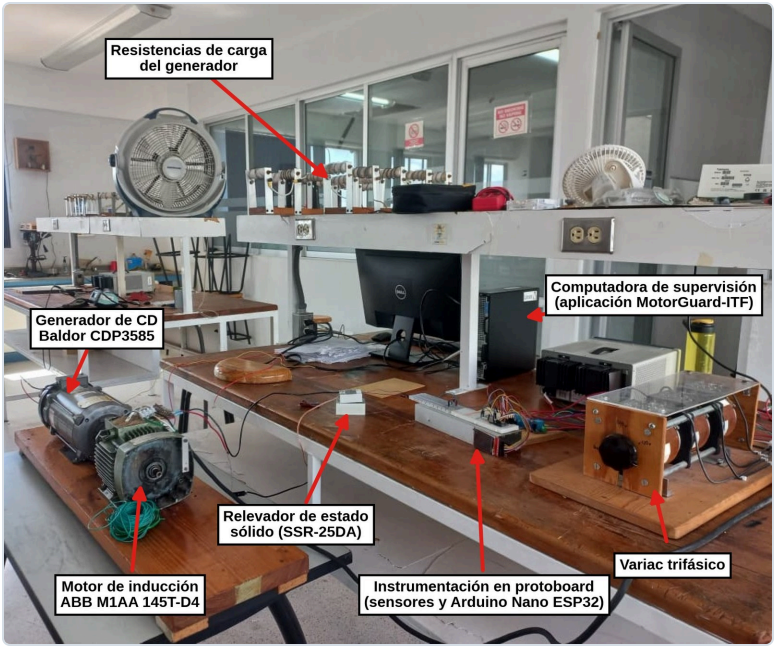
PORTFOLIO · THESIS PROJECT · UASLP 2025–2026

# Inter-turn short-circuit detection in induction motors

**Juan Emmanuel Luna Velázquez**
Mechatronics Engineering graduate
Matehuala, S.L.P., Mexico · linkedin.com/in/juan-emmanuel-luna
juanemmanuel2220@gmail.com

An inter-turn short circuit in a stator winding starts small and, if undetected, ends up burning the motor. I built a complete system that detects it **by measuring only the supply currents and voltages**, without touching the motor: from the test bench and real-time firmware to machine learning analysis and a desktop app with a local AI assistant.

## THE TEST BENCH




Test bench at the UAMRA laboratory (labels in Spanish).

**1.5 kW**
three-phase ABB induction motor, supplied at 123 V per phase

**0–75 %**
load, set with a DC generator and a resistor bank

**3 severities**
of fault: 7.0, 14.3 and 21.6 % of the turns of one phase

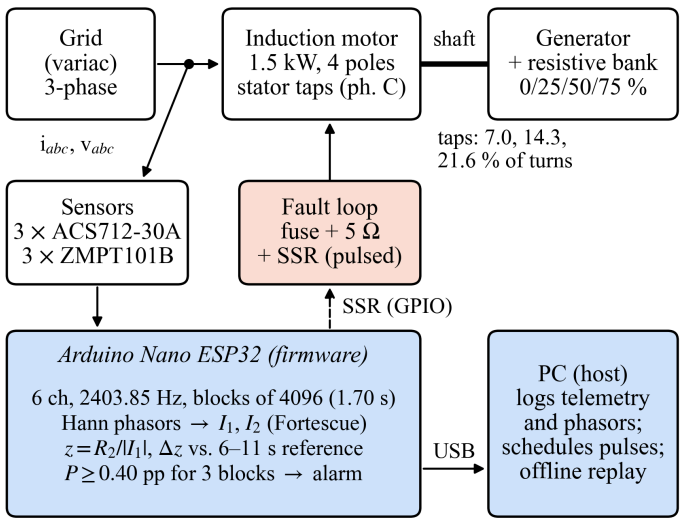
**6 channels**
3 ACS712 current sensors and 3 ZMPT101B voltage sensors

**14,425**
blocks processed over 6.8 h in a row without losing one

**597**
healthy/fault cases captured on the bench for training and validation

The fault is emulated with winding taps and a solid-state relay that connects it in short pulses, with a fuse and a 5 Ω limiting resistor to protect the motor.

## SYSTEM ARCHITECTURE




The ESP32 makes the detection decision on its own; the PC logs telemetry, schedules the fault pulses and runs the assistant.

## TOOLS

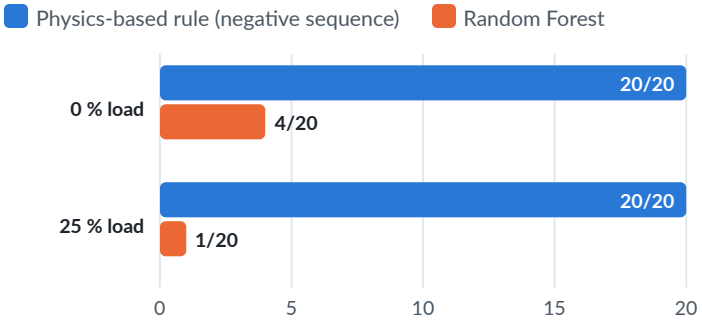
ESP32 · C/C++ | ACS712 · ZMPT101B | Symmetrical components (Fortescue) | Python · scikit-learn | Statistics (Wilson, Poisson, McNemar) | PySide6 · pyqtgraph | llama.cpp · RAG | git · Linux · Docker · pytest

## WHAT EACH PART DOES

- **C/C++ firmware (ESP32):** samples 6 channels at 2,403.85 Hz in blocks of 4,096 samples (1.70 s), computes the phasors and the **negative-sequence current** with Fortescue's symmetrical components, and applies a detection rule with 3-block persistence and alarm latching. It shows the state on a TFT display and streams telemetry over USB.
- **Python analysis:** 597-case dataset; Random Forest, Gradient Boosting, logistic regression and decision tree, validated by leaving whole groups out (unseen loads). Analysis protocols registered in git before analysis, and blind validations.
- **Desktop app (PySide6):** real-time monitoring, safe control of fault injection, and a local RAG assistant (llama.cpp) that recommends actions citing maintenance manuals from ABB, NASA, DOE and USBR.

## KEY RESULT: THE AI DID NOT GENERALIZE, THE PHYSICS DID

**Mild faults (7.0 %) detected in the blind validation**, out of 20 pulses per load

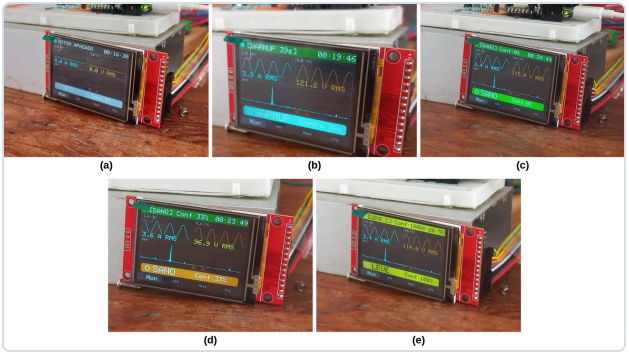



The Random Forest looked very good in cross-validation and had **fewer false alarms** (2 vs. 21 for the rule over 4.9 h). But at low loads it missed almost all mild faults: it had learned a **shortcut**, a feature that depended on the load rather than on the fault.

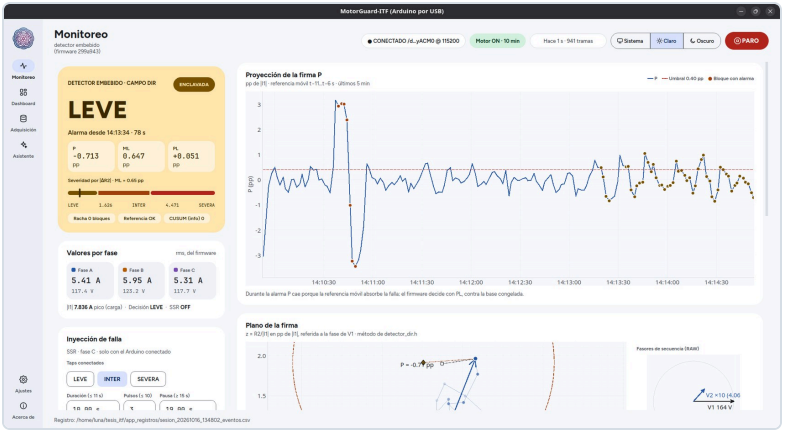
The physics-based rule, built on the negative-sequence current, detected all of them. So I redesigned the system: **the rule does the detection** on the microcontroller, and AI is used where it did work: the **severity classifier was correct in 596 of 597 cases** at unseen loads, and the assistant turns the alarm into a recommendation.

**What did not work (and I reported it):** in a 90-minute healthy-motor test, the rule produced 7.3 false alarms per hour (95 % CI: 3.7–13.1), far from the target of 1 per hour. The cause is slow noise in the bench itself; increasing the persistence lowers it, but that needs a new validation campaign. It is the next problem to solve.

## WHAT THE OPERATOR SEES




ESP32 TFT display: (a) motor off, (b) warm-up, (c) and (d) healthy motor, (e) mild fault detected.




Desktop app (Spanish UI) with a latched mild-fault alarm: fault signature over time, per-phase values and injection control.

## MY ROLE

- Built and wired the bench, took the measurements and ran the capture campaigns.
- Defined the detection rule, the protocols and the acceptance criteria before looking at the results.
- Programmed the firmware, the analysis and the app, using AI coding assistants to speed up the code; the technical decisions and the validation were mine.
- Wrote the thesis (144 pages) and a manuscript in IEEE format with my advisors.

### What I learned

- Validate under conditions the model has not seen: good average accuracy can hide a shortcut.
- Design the bench with safety in mind: fuse, limiting resistor and short pulses.
- Report what does not work as clearly as what does.